

IN-PLACE MERGE SORT

Of compromises and the elegance lie

Florian Sihler

September 21, 2026
SP, Ulm University

IN-PLACE MERGE SORT

Of compromises and the elegance lie

Florian Sihler



Officially supported by the Ping-Ping-Uh-Uh-Foundation for Emotional Peeps. Peep n' peace and wa... oh look, there's cake!

September 21, 2026
SP, Ulm University

2

Merge Sort itself

```
def sort(list: List) → List:
  if len list ≤ 1 : ret list;
  left, right ← split(list);
  ret merge(sort(left), sort(right));
```

```
def merge(a: List, b: List) → List:
  merged ← empty List;
  while a & b aren't empty :
    x ← a[0] ≤ b[0] ? a : b;
    merged.add(x.remove(0));
  merged.add(a not empty ? a : b);
  ret merged;
```

We will only look at the ascending case. An example for the hordeeeee...



[4]: BTPK-Package, tikzpingus
Sihler, 2021



3

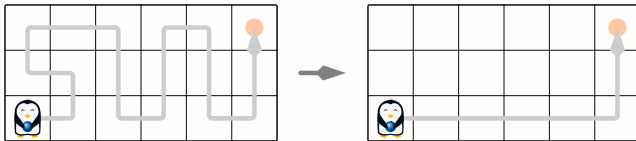
In-Place

- The meaning varies:
 - Constant additional memory: $\mathcal{O}(1)$.
possibly also thanks to compiler optimizations
 - Constant additional pointer/index accesses
 - Constant memory to manipulate the input
 - ...
- If not in-place: “out-of-place”

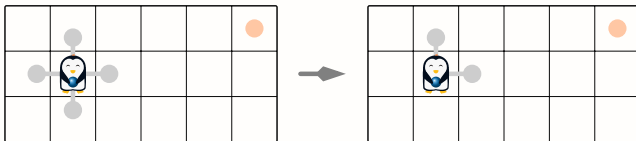
4

Nothing is free

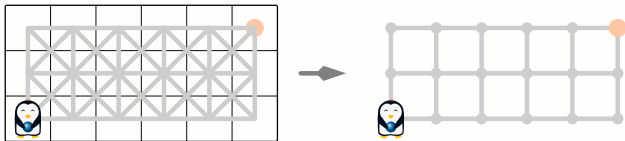
- We improve the runtime by:
 - removing redundant operations



- exploiting the constraints (e.g. numbers only)



- We improve the runtime by:
 - abstracting cleverly (e.g. reduction)



- additional memory (e.g. as a cache)

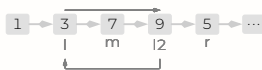
- We improve the runtime by:
 - removing redundant operations
 - exploiting the constraints
 - abstracting cleverly
 - additional memory
- Most common compromise: memory vs. runtime

Pointer Comparison

Now the time complexity has gotten worse.
From $\mathcal{O}(n \log n)$ to $\mathcal{O}(n^2)$.



```
void merge(int[] arr, int l, int m, int r) {
    int l2 = m + 1;
    if(arr[m] <= arr[l2]) return;
    while(l <= m && l2 <= r) {
        if(arr[l] <= arr[l2]) { l++; }
        else { // shift elements from l, 1 to right
            int shift = arr[l2], idx = l2;
            while(l < idx) {
                arr[idx] = arr[idx - 1]; idx--;
            }
            arr[l] = shift;
            l++; m++; l2++;
        }
    }
}
```





```
void sort(int[] arr) {  
    int n = arr.length - 1;  
    for (int len = 1; len <= n; len = 2*len) {  
        for (int left = 0; left < n; left += 2*len) {  
            int mid = Math.min(left + len - 1, n);  
            int right = Math.min(left + 2*len - 1, n);  
            merge(arr, left, mid, right);  
        }  
    }  
}
```

1. Each merge grows the list length faithfully by 2^i .
2. We can start directly with single-element lists.



9

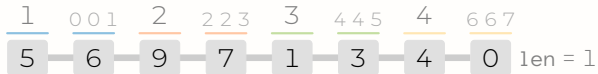
Iterative Split



In case the array would need only one merge, or none at all, to begin with.

The three small, light gray numbers mark the indices of left, mid and right.

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



Classics



A Better Merge



A Better Split



Conclusion



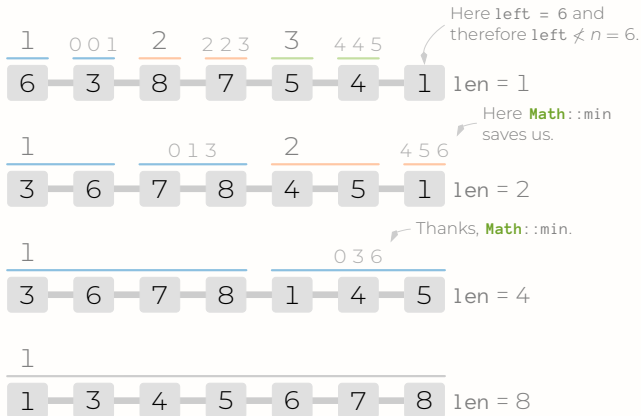
10

Iterative Split



What happens if the length is not a power of two?

```
void sort(int[] arr) {
    int n = arr.length - 1;
    for (int len = 1; len <= n; len = 2*len) {
        for (int left = 0; left < n; left += 2*len) {
            int mid = Math.min(left + len - 1, n);
            int right = Math.min(left + 2*len - 1, n);
            merge(arr, left, mid, right);
        }
    }
}
```



What remains to be noted...

- The additional memory is now entirely in $\mathcal{O}(1)$
- But the runtime is now in $\mathcal{O}(n^2)$
 - The iterative sort contributes “only” $\mathcal{O}(n \log n)$ to that
 - The complexity springs from the merge...

- (Presumably) there is nothing below $\mathcal{O}(n \log^2(n))$
- That bound is reached with a Shell-Sort-inspired approach
- There are many more derivatives, such as the “Natural Mergesort”...

[2]: *Efficiently merging two sorted arrays with $\mathcal{O}(1)$ extra space*, 2021

[5]: *The algorithm design manual*
Skiena, 2020

[3]: *Merge path-parallel merging made simple*
Odeh, 2012

[1]: *Sublinear merging and natural mergesort*
Carlsson, 1993

- [1] Svante Carlsson, Christos Levkopoulos, and Ola Petersson. “Sublinear merging and natural mergesort”. 1993
- [2] *Efficiently merging two sorted arrays with $O(1)$ extra space*. 2021
- [3] Saher Odeh et al. “Merge path-parallel merging made simple”. 2012
- [4] Florian Sihler. *L^AT_EX*-Package, *tikzpingus*. 2021
- [5] Steven S. Skiena. *The algorithm design manual*. 2020

Florian Sihler

Ulm, September 21, 2026

